

Business Logic Vulnerabilities in Web and Mobile Applications

Ahmet Eren Arslan

Introduction

Today, security in web and mobile applications is not limited only to technical vulnerabilities. Alongside classic vulnerabilities such as SQL Injection, XSS, or weak authentication, logic flaws arising from the application's business workflow can also create serious security risks.

What Is a Business Logic Vulnerability?

A Business Logic vulnerability is a design or logic flaw that may cause a system to become vulnerable to attacks during its design and implementation.

I can explain this with an example from my daily life: I earned the right to eat for free in our school's cafeterias. Therefore, when I scan my student card at the cafeteria, I can pass through without paying. There are three cafeterias at our school. Under normal circumstances, each student should consume only one meal at school per day. However, by using this free-pass right granted to me in all three different cafeterias at our school, I can consume three free meals in one day. In a bad scenario, someone could take advantage of this card vulnerability and rent the card to a person who normally pays for cafeteria meals at a price lower than the cafeteria fee. This is a good example of a Business Logic vulnerability that I have encountered in my own life.

Business Logic Vulnerabilities in Web and Mobile Applications

The situation is actually quite similar in web and mobile applications.

Business Logic vulnerabilities can appear in web and mobile applications in situations such as creating an order in e-commerce without completing payment, price manipulation, repeated or inappropriate coupon usage, incorrect monetary amounts such as negative values in banking applications, failure to perform capacity checks in reservation-based applications, and similar cases.

Most Common Business Logic Vulnerabilities in Web and Mobile Applications

1. **Price Manipulation:** This is a vulnerability where an attacker can modify the price using client-side parameters.
2. **Account Takeover Through Password Reset Flaws:** These are vulnerabilities that may allow an attacker to abuse password reset methods in a way that enables access to another user's account.
3. **Exploiting Quantity Limits:** These vulnerabilities occur when a user bypasses restrictions related to the number of items they can purchase, use, or access.
4. **Privilege Escalation Through Flawed Role Management:** These vulnerabilities occur when inappropriate permissions are assigned to user roles.
5. **Multi-Step Flow Manipulation:** These are vulnerabilities that may allow attackers to skip multi-step processes such as order confirmation or payment approval.

These are among the most commonly encountered Business Logic vulnerabilities in web and mobile applications.

Historical Business Logic Vulnerabilities in Web and Mobile Applications

1. **Stripe Unlimited Discount Usage:** A user on HackerOne discovered that a payment discount in Stripe could be applied multiple times to the same account. (HackerOne)
2. **Starbucks Gift Card Race Condition Incident:** During the period when Starbucks gift cards were introduced, the same balance transfer transaction between gift cards could be triggered twice simultaneously. (SecurityWeek)
3. **Uber Free Ride Bug:** A user named Anand Prakash discovered that when invalid payment method values such as `abc` or `xyz` were entered in Uber, the ride could be completed without being charged. (TechCrunch)

Example Applications

Price Manipulation

A normal request to add a product to the cart may look like this:

```
POST /api/cart/add HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "product_id": 1024,
  "quantity": 1,
  "price": 1500
}
```

An attacker can intercept this request and modify the price value:

```
POST /api/cart/add HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "product_id": 1024,
  "quantity": 1,
  "price": 1
}
```

If the server trusts the value received from the user instead of verifying the price information from its own database, the product can be purchased for an amount much lower than its real price.

2FA Logic Bypass

Let us assume that Two-Factor Authentication, or 2FA, is used in a login process. The normal flow proceeds as follows:

1. The user enters their username and password.
2. The server verifies the credentials and redirects the user to the `/login/otp` page.
3. The user enters the OTP code.
4. The server verifies the OTP code and redirects the user to the `/dashboard` page.

A logic flaw in this flow may occur if the server gives the user an authenticated session cookie immediately after the first step, but only tries to prevent the user from seeing the dashboard page through the user interface. In such a scenario, after logging in with a username and password, the attacker may manually try to access the `/dashboard` address without completing the OTP step. If the 2FA check is not performed on the backend side, the attacker can completely bypass the OTP requirement.

In a similar case, the application may track the 2FA status using a boolean flag inside the session:

```
// Vulnerable server-side logic - Pseudocode
if (user.checkPassword(pass)) {
    session.user_id = user.id;
    session.needs_2fa = true;
    res.redirect('/otp-page');
}

// Dashboard check
if (session.user_id) {
    // Missing check for session.needs_2fa == false
    res.render('dashboard');
}
```

In this example, the developer forgot to check whether the `needs_2fa` value is `false` on the dashboard route. Therefore, the user may behave as if they are logged in by passing only the password stage. In this type of vulnerability, the problem is not the absence of an OTP mechanism, but the failure to enforce the OTP check on critical endpoints.

Infinite Discount Loop

Some applications allow users to apply multiple coupon codes. However, if the system does not check whether the total discount exceeds the total price, a Business Logic vulnerability may occur.

For example, suppose an attacker finds a 10-dollar discount coupon named `WELCOME10`. If the application allows the same coupon code to be applied repeatedly, or if it allows different coupons to accumulate until the total price drops below zero, the attacker may obtain products for free or abuse the application's discount/balance logic.

A repeated coupon request may look like this:

```
POST /cart/apply-coupon HTTP/1.1
Host: example-store.com

coupon_code=WELCOME10&coupon_code=WELCOME10&coupon_code=WELCOME10
```

If the backend calculation is performed as follows:

```
Total = Total - (CouponValue * Count)
```

and the `Count` value is not limited to 1, then the application's discount logic is broken. In this case, because the system does not prevent the same coupon from being used more than once, the total amount may be calculated incorrectly.

Impact Analysis

Unlike software or hardware vulnerabilities, these vulnerabilities can directly affect company data and revenue.

- **Financial Loss:** They can cause financial damage directly through methods such as discounts and coupon usage.

- **Data Breaches:** They may enable access to other users' data through unauthorized access.
- **Reputational Damage:** They can cause serious harm to a company's reputation and reliability.
- **Operational Disruption:** Attackers may cause interruptions in other users' access in order to exploit the vulnerabilities.

Prevention Methods

- Values that are critical from a security perspective should be regenerated on the server side. For values such as price, authorization, ownership, and transaction status, client-side data should only be accepted as input and should not be treated as the source of truth.
- Business workflows should be implemented with a clear state machine logic. The order of steps should be enforced not only through the user interface but also on the backend side.
- Concurrency should be treated as a real threat. If it is possible for two requests to run at the same time, it should be assumed that an attacker will try to exploit this.
- Rate-limiting and monitoring controls should be applied not only at the authentication level but also at the feature level. Features that are open to abuse, such as referral systems, coupon usage, and password reset mechanisms, should have their own controls.
- Threat modeling should be performed not only based on the technical architecture but also based on the business process. It should be evaluated not only what happens when an attacker sends a malicious payload, but also how the system reacts when the user does not behave honestly.

Conclusion

Business Logic vulnerabilities occur when business rules are implemented incorrectly even though the application technically works. This creates vulnerabilities that may expose the system to external attacks or unwanted incidents. Security vulnerabilities are not only issues that arise from hardware or software. Therefore, in security testing, importance should be given not only to automated tools but also to manual analysis and business workflow testing.

References

- <https://portswigger.net/web-security/logic-flaws>
- <https://bitpanic.medium.com/top-10-overlooked-business-logic-flaws-in-web-applications-6745d42f3786>

- <https://owasp.org/www-project-top-10-for-business-logic-abuse/>
- <https://blogs.jsmon.sh/what-is-business-logic-vulnerability-ways-to-exploit-examples-and-impact/>
- https://cheatsheetseries.owasp.org/cheatsheets/Business_Logic_Security_Cheat_Sheet.html